Outputs panel

Files an agent makes mid-run, visible the second they exist — plus a starting point for users who open the Playground and don't know what to ask for.

OWNER	AUDIENCE	STATUS	SCOPE
Mk	Design + Engineering	Ready to build (3 decisions in §8)	Simple edit to the Playground, one flag

PATTERN REFERENCE
ChatGPT "Outputs" side panel

§5 Designers start here
§6 Engineers start here
§8 Decisions for Mk

§1

A file made 23 minutes into a run is invisible until the run ends. Fix the plumbing, then add the panel

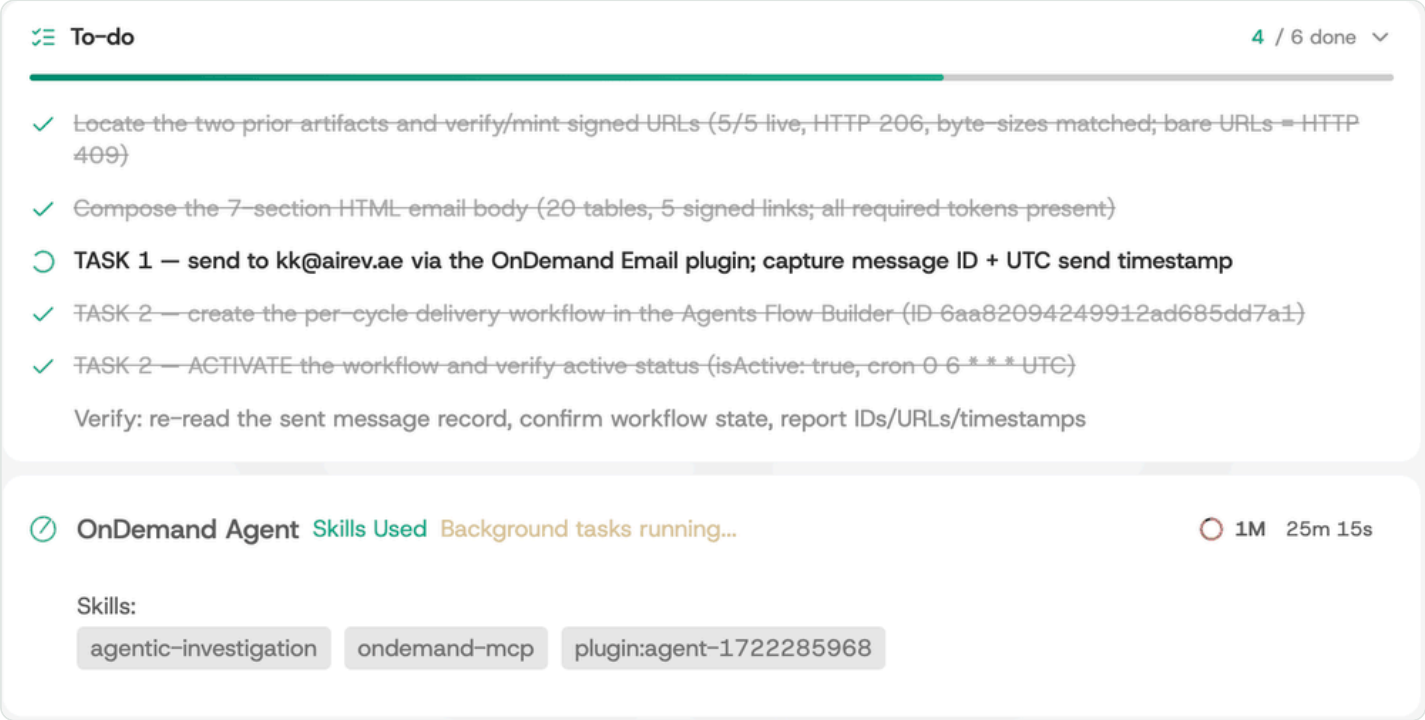
- **Problem:** the Playground only surfaces an agent's files when the run finishes (or via the transcript, which scrolls away). A 25-minute run that wrote a PDF at minute 23 shows nothing but "Background tasks running…"
- **Fix:** register every file the moment it is written (`artifact.created` event + DB row), and show a run-scoped **Outputs** panel in the right rail that upserts live. Refresh-safe, stop-safe, openable mid-run
- **Plus:** a **+** on the panel opens a templates tray and pre-fills the composer ("Create a new document with *Documents*. Start by asking me what it should be about.") — same pattern as ChatGPT, OnDemand palette
- **Size:** ~10–14 working days across 1 BE, 1 FE, 1 designer in parallel (§7). Backend event change is 80% of the value and ~20% of the work
- **Open decisions:** right rail vs left nav, "Outputs" vs "Artifacts" naming, default-open breakpoint (§8)

THE ONE CHANGE

Stop tying artifact registration to run completion. Register at write time, stream the event, persist the row. Everything else in this document follows from that.

§2 · TODAY

What the user sees at minute 25: a to-do list and a timer, not the document that already exists



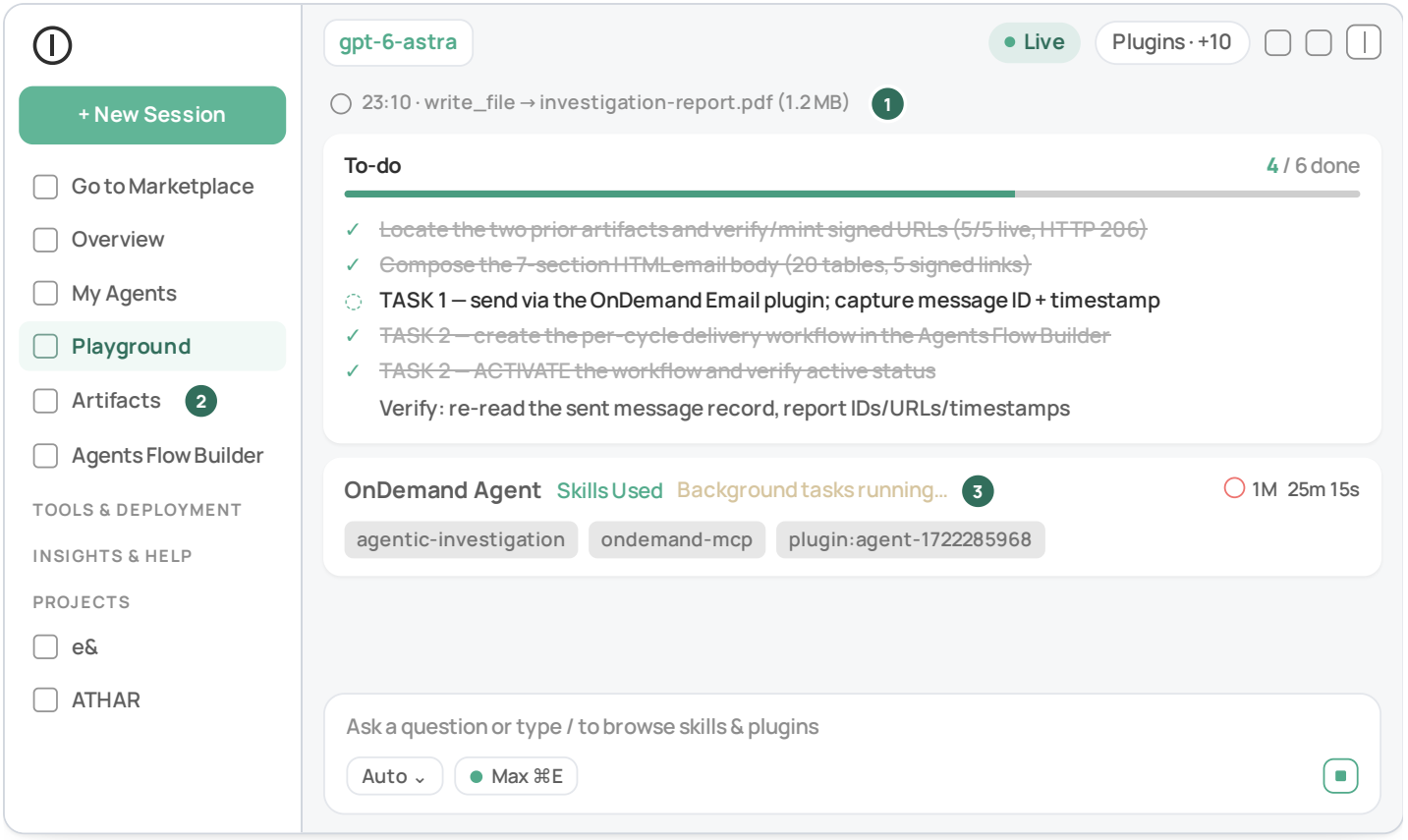
Your run, 25m 15s in. The agent has already produced a PDF report and an XLSX workbook (the to-do list references both). Nothing on screen lets the user open either. The only trace is a tool line in the transcript, which scrolls out of view within a minute.

- **Where files go today:** written to the run workspace; a DB row and a link appear only after `finalizeRun()`; the left-nav **Artifacts** page lists them after that
- **Three failure modes:** (a) long runs — user waits 20+ min for a file that was ready at minute 5; (b) stopped runs — user hits ■ and loses sight of everything produced; (c) crashed runs — files exist on disk, no row, effectively gone
- **Cost:** the product's best moments (a finished IC memo, an evidence workbook) happen off-screen. Users re-run, or export via email as a workaround — which is exactly what the run in the screenshot is doing

§3 · TARGET

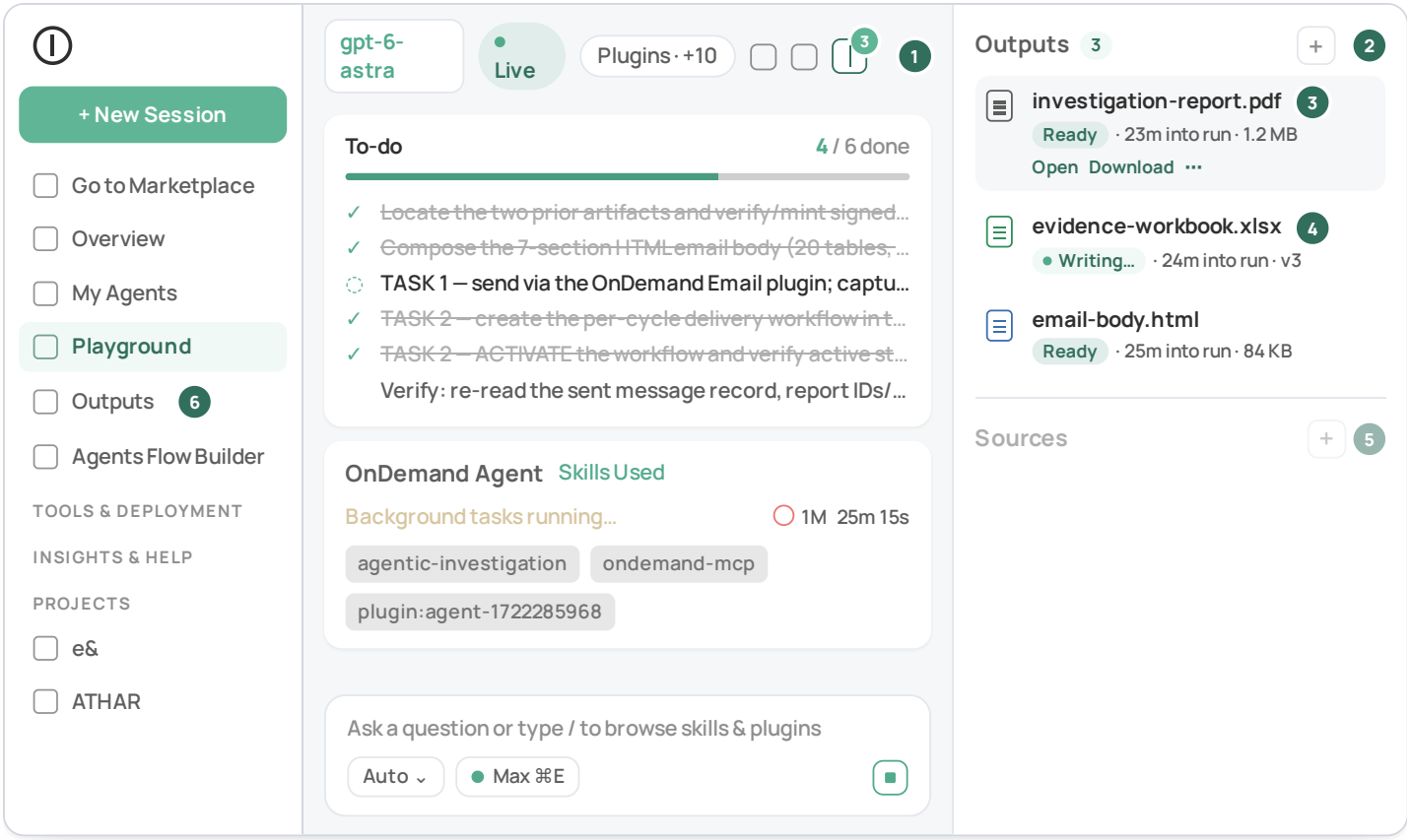
Outputs appear the second they exist, open without interrupting the run, and "+" gives confused users a starting point

Mockups below are illustrations in OnDemand's sampled palette (§5.3), not pixel-final. Numbered markers map to the notes under each frame. Scroll sideways on a phone.



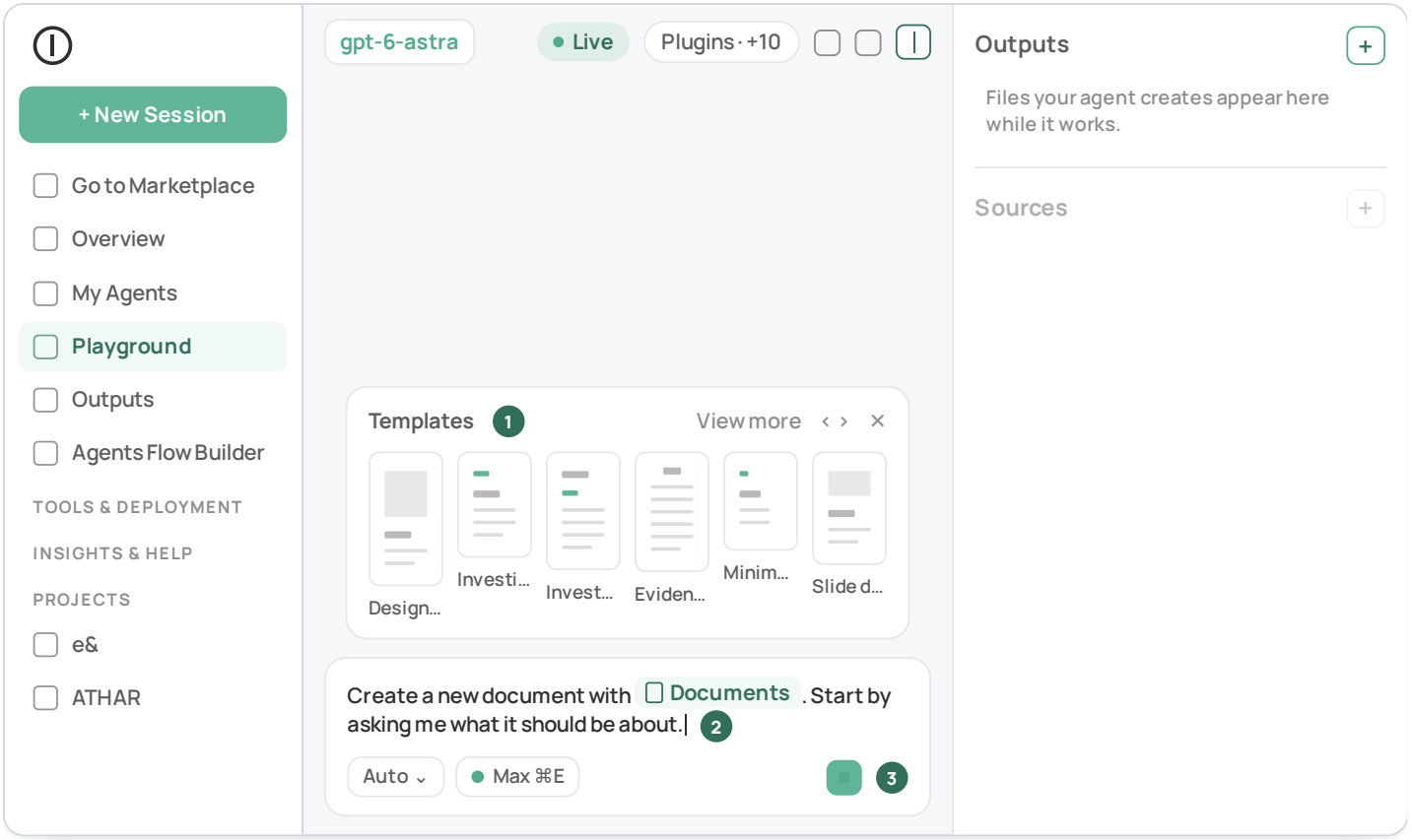
Before — today.

1. The only evidence a file exists: one muted tool line in the transcript. It scrolls off within a minute and isn't clickable
2. "Artifacts" in the left nav is a global library that only lists files after `finalizeRun()`. Mid-run it shows nothing for this session
3. The only live signal is the status line and the timer. The user has no way to tell whether 25 minutes has produced anything



After — mid-run, same second. The panel fills as the agent works; the transcript is untouched.

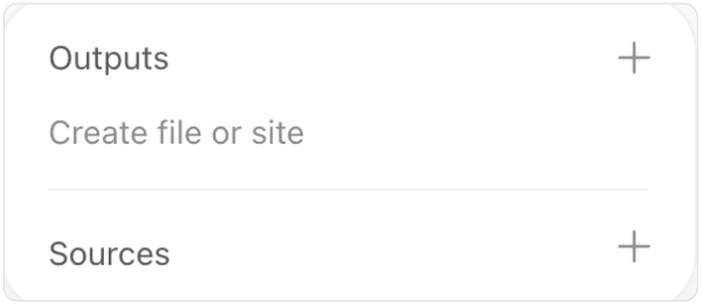
1. Toggle lives in the existing top-right icon slot. Collapsed, it carries a green badge with the count of outputs the user hasn't seen yet
2. Header: "Outputs" + count + **+**. The + opens the templates tray and pre-fills the composer (next frame)
3. **Ready** = written and openable now, run keeps going. Meta line: status · minutes into the run · size. Hover/focus reveals Open · Download · ···
4. **Writing...** = the agent is still rewriting this file (version bumps in place, no duplicate rows). Opening it shows the last complete version
5. **Sources** mirrors ChatGPT's second section — phase 2, ship greyed or hidden behind the flag
6. Left-nav "Artifacts" becomes "Outputs" (all runs). One word everywhere — decision in §8



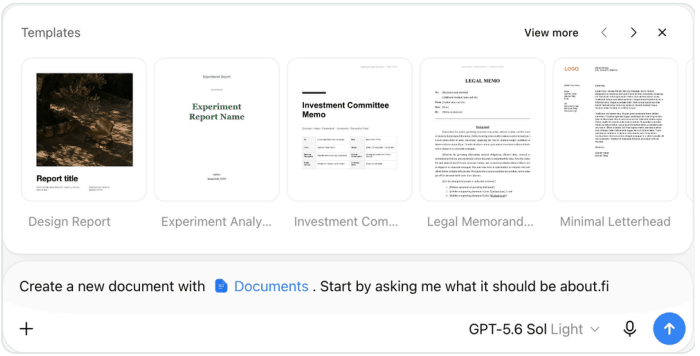
After — "+" pressed on an empty panel. Nothing is sent until the user hits send; the tray and the pre-filled prompt remove the blank-page problem.

1. Templates tray slides up above the composer: six cards with *real rendered thumbnails* of example outputs, "View more", arrows, close. Clicking a card swaps the composer text for that template's prompt
2. Composer pre-filled with a skill chip (brand-50 background, brand-700 text — the green equivalent of ChatGPT's blue "Documents" chip). Caret at the end so the user can just type
3. Send button turns solid brand green because the composer now has content. Send starts a normal run; the first file lands in the panel as before

Copy ChatGPT's pattern, not its look



ChatGPT — Outputs panel, resting state. Two sections, each with a "+". Title #5d5d5d, secondary #8f8f8f, divider #f2f2f2, "+" border #e3e3e3, on white.



ChatGPT — after "+". Tray above the composer, real thumbnails, and a pre-filled prompt with a blue skill chip (#4e82ef). Nothing sent yet.

COPY	DON'T COPY
One-word header "Outputs", "+" top-right, count next to the word	Blue accent — use brand green (#52aa8b / #326e5c)
"Sources" as a second section with its own "+"	#8f8f8f secondary text (3.2:1 on white, fails AA) — use #5d5d5d
Templates tray above the composer: thumbnails, "View more", arrows, ✕	Generic prompt wording — keep the shape, name OnDemand's skill
Composer pre-filled with a skill chip; nothing sent until the user presses send	Rows with no status — ChatGPT has no notion of a file still being written; ours needs Writing… / Ready / Final / Partial
Panel is per-conversation, collapsible, stays open across turns	Its ~340px width at 2000px viewport — use 320px and collapse by default below 1280px

Same Playground, one new surface: a 320px right-rail panel that uses tokens the UI already has

5.1 Placement

- **Right rail**, 320px wide, full height of the transcript column, own scroll. Border-left 1px line-1. The transcript column shrinks; the left nav is untouched
- **Toggle** in the existing top-right icon slot (the collapsed-rail glyph already there). Collapsed state carries a badge with the unseen count
- **Default:** open at ≥1280px viewport when the run has ≥1 output; collapsed below 1280px. Remember the user's last choice per browser
- **Scope:** the active session's run(s). "All outputs" link at the bottom → left-nav library

5.2 Anatomy

PART	SPEC
Panel	320 × full height · padding 16 · gap 12 · background surface-0 · border-left 1px line-1
Header	"Outputs" 15/600 ink-700 · count pill 12/500 brand-700 on brand-50, radius 999 · "+" 28×28, radius 8, 1px line-1, glyph ink-500; hover border brand-500, glyph brand-700
Row	grid 20px · 1fr · auto; min-height 44; padding 8; radius 10; gap 10. Hover / focus-within: surface-1. Whole main area is one button (opens preview); actions are separate buttons
Kind icon	20px, 1.5px stroke, ink-700. Set: document, sheet, deck, site, image, other. Colour-tint only for sheet (green) and site (blue) — matches OS conventions users already know
Name	13/500 ink-900, single line, middle-ellipsis so the extension stays visible
Meta line	12/400 ink-700: status pill · "23m into run" · size (· "v3" while writing)
Status pills	12/500, padding 1px 7px, radius 999. **Writing…** brand-50 bg, brand-700 text, 6px pulsing brand-500 dot · **Ready** brand-100 bg, brand-700 text · **Final** chip-bg, ink-700 text · **Partial** warn-tint bg, warn-text
Row actions	On hover/focus: Open · Download · ⋯ (Copy link, Open in Outputs library). 12/600 brand-700 text buttons, no borders
Sources	Second section, same header pattern, 1px line-1 divider above. Phase 2 — ship greyed (opacity .5) or hidden
Empty state	One line, 13/400 ink-500: "Files your agent creates appear here while it works." No illustration
Collapsed badge	15px min, radius 999, brand-400 fill, white 10/800 number, 2px white ring. Resets to 0 when the panel opens

5.3 Colour tokens — sampled from your screenshot, with contrast checked

Values come from a Retina screenshot of the live Playground, so expect ±1–2 hex steps from the source tokens. Engineers: map to the existing token names; the proposed names below are for this document only. Contrast is against #ffffff unless stated.

TOKEN	HEX	SAMPLED FROM	USE IN THE PANEL	CONTRAST
brand-500	#52aa8b	Live dot, Max dot, ✓ marks, spinner, "Skills Used", model name	Writing… dot, focus ring, "+" hover border, toggle-on stroke. **Never for text**	2.8:1
brand-400	#61b597	"New Session" button	Collapsed badge fill, active send button	— (fill)
brand-600	#4a9c80	To-do progress fill	Not used	—
brand-700	#326e5c	Active nav label	Pill text, count text, chip text, row-action text	6.0:1
brand-50	#f1f9f6	Active nav bg, active session row	Writing… pill bg, count bg, composer skill-chip bg	5.6:1 w/ brand-700
brand-100	#e0edea	"Live" pill bg	Ready pill bg	5.0:1 w/ brand-700
ink-900	#2e2e2e	To-do title, plugin count	Output name	13.6:1
ink-700	#5d5d5d	"OnDemand Agent", timer	Panel title, meta line, Final pill text, kind icons	6.6:1
ink-500	#8d8d8d	Transcript body text, section labels	"+" glyph and empty-state line only — **not for meta text**	3.3:1
ink-300	#aeaeae	Done to-do items (struck)	Not used in the panel	2.2:1
surface-0	#ffffff	Sidebar, cards, composer	Panel background	—
surface-1	#f6f7f8	Main canvas	Row hover / focus-within	—
line-1	#e4e7ec	Composer border, right-rail border	Panel border, dividers, "+" border, tray card border	—
line-2	#d1d6de	Sidebar divider	Not used	—
chip-bg / chip-text	#e7e7e7 / #737373	Skill chips	Final pill bg — but text ink-700, not #737373	3.8:1 as-is
warn / warn-text	#d6c59f / #7a5f1c	"Background tasks running…"	Partial pill: dot #d6c59f, bg #fbf5e6, text #7a5f1c	5.5:1 text on bg
danger	#ed7470	"1M" context ring	Failed-write icon tint only	2.9:1

Rule that falls out of the numbers: brand green is a dot, ring and fill colour, never a text colour — brand-700 carries every green word. Status is never colour-only; every pill has a label.

5.4 Type, spacing, motion

- **Type:** the Playground's existing UI face, no new family. Title 15/600 · name 13/500 · meta 12/400 · pill 12/500 · tray card title 13/500 · section label 12/600 uppercase, +0.06em
- **Spacing:** 4-pt grid. Panel padding 16 · row gap 4 · section gap 12 · pill padding 1×7 · tray padding 16, card gap 12, cards 148×186, radius 12
- **Motion:** new row fades + slides up 4px over 160ms ease-out · Writing… dot pulses 1.2s · tray slides up 200ms · all off under `prefers-reduced-motion`
- **Dark mode:** panel surface #1a201d, border #2a322e, name #e8ece9, meta #b9c1bd, pills keep brand-700-on-tint contrast ≥4.5:1 (re-derive tints on the dark ground; don't invert)

5.5 Copy — final wording

WHERE	TEXT
Panel header	Outputs
Empty state	Files your agent creates appear here while it works.
Pills	Writing… · Ready · Final · Partial
"+" tooltip	Create an output
Pre-filled composer	Create a new document with *[Documents]*. Start by asking me what it should be about.
Toast (panel collapsed)	New output: investigation-report.pdf — Open
Screen reader (live region, polite)	New output ready: investigation-report.pdf
Partial tooltip	Run stopped before this file was finished. Last saved version shown.
Footer link	All outputs →

5.6 Designer to-dos

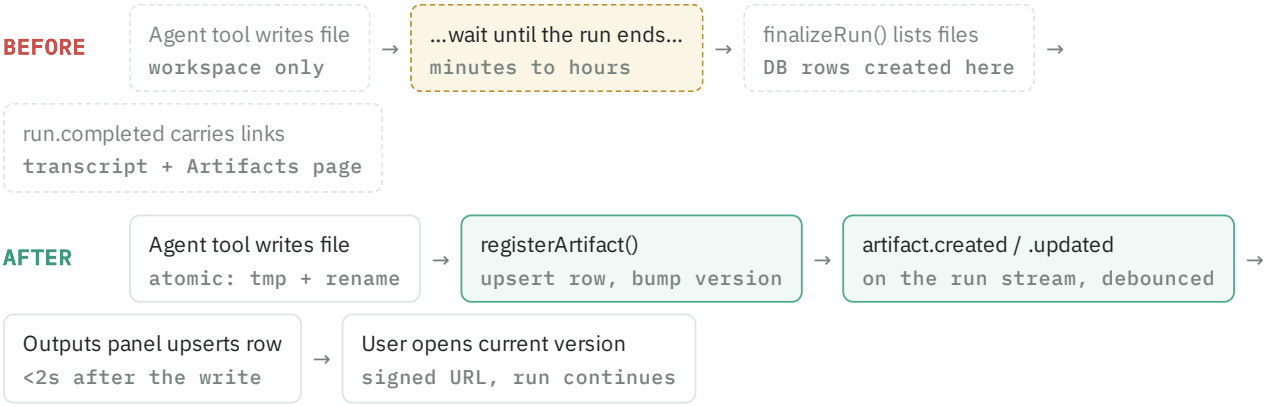
- [ ] Add the panel to the Playground master frame in Figma at both states (collapsed with badge / expanded), desktop 1440 and 1280
  Both light and dark
- [ ] Row component with 4 status variants × hover / focus-within / pressed, plus the actions row
- [ ] Kind icon set: document, sheet, deck, site, image, other — 20px, 1.5px stroke, on the same optical grid as the existing nav icons
- [ ] Empty state, templates tray (6 cards, "View more", arrows, ✕), composer chip and filled send button

- [ ] Six template thumbnails rendered from real example outputs (not grey boxes) — 296×372 @2x, PNG
  Investigation report · IC memo · Design report · Evidence workbook · Slide deck · Minimal letterhead
- [ ] Motion spec: row enter, dot pulse, tray slide, badge increment — with reduced-motion fallbacks
- [ ] Map every colour in §5.3 to the codebase's token names with engineering; flag any that don't exist
- [ ] Rename "Artifacts" → "Outputs" in the left nav and the library page header (pending §8 decision)

Register at write time, stream the event, persist the row. The panel is a subscriber

Code is illustrative TypeScript/React against a generic store and stream — adapt names to the Playground's modules. Every snippet is a before/after of one seam; the seams are what matter.

6.1 The data path



6.2 Data model

MIGRATION — db/migrations/2026_09_outputs_panel.sql

```sql
-- One row per (run, path). Rewrites bump `version` in place; nothing is ever deleted.
ALTER TABLE artifacts
  ADD COLUMN status        TEXT     NOT NULL DEFAULT 'ready',   -- writing | ready | final | partial
  ADD COLUMN version       INTEGER  NOT NULL DEFAULT 1,
  ADD COLUMN content_hash  TEXT,                                -- sha256; same hash => no event
  ADD COLUMN produced_by   TEXT,                                -- tool | sub-agent | background-task id
  ADD COLUMN ms_into_run   INTEGER;                             -- "23m into run"

CREATE UNIQUE INDEX artifacts_run_path    ON artifacts (run_id, path);
CREATE INDEX        artifacts_run_created ON artifacts (run_id, created_at);

-- Backfill: everything that exists today was produced by a finished run.
UPDATE artifacts SET status = 'final' WHERE status = 'ready';
```

6.3 Event contract

BEFORE — shared/run-events.ts

```ts
// What the Playground stream carries today.
export type RunEvent =
  | { type: 'run.started';   runId: string }
  | { type: 'todo.updated';  items: TodoItem[] }
  | { type: 'skill.used';    name: string }
  | { type: 'message.delta'; text: string }
  | { type: 'run.completed'; artifacts: ArtifactRef[] } // files surface here...
  | { type: 'run.failed';    error: string };           // ...or never
```

AFTER — shared/run-events.ts

```ts
// Outputs get their own events, independent of run state.
export type ArtifactStatus = 'writing' | 'ready' | 'final' | 'partial';

export interface Artifact {
  id: string;
  runId: string;
  path: string;              // workspace path, unique per run
  name: string;              // "investigation-report.pdf"
  kind: 'pdf' | 'docx' | 'xlsx' | 'pptx' | 'html' | 'md' | 'csv' | 'image' | 'other';
  bytes: number;
  version: number;           // bumps on every content change
  status: ArtifactStatus;
  producedBy: string;        // tool / sub-agent / background task id
  msIntoRun: number;         // -> "23m into run"
  createdAt: string;
  updatedAt: string;
}

export type RunEvent =
  | { type: 'run.started';        runId: string }
  | { type: 'todo.updated';       items: TodoItem[] }
  | { type: 'skill.used';         name: string }
  | { type: 'message.delta';      text: string }
  | { type: 'artifact.created';   artifact: Artifact }    // NEW
  | { type: 'artifact.updated';   artifact: Artifact }    // NEW
  | { type: 'artifact.finalized'; artifactIds: string[] } // NEW
  | { type: 'run.completed' }                             // no file list any more
  | { type: 'run.failed';         error: string };
```

6.4 Registration — the change that does the work

BEFORE — server/run-lifecycle.ts

```ts
// TODAY: files are collected once, when the run ends.
export async function finalizeRun(run: Run) {
  const files = await workspace.list(run.id, { since: run.startedAt });
  const refs  = await db.artifacts.createMany(
    files.map(f => toArtifactRef(run, f)),
  );
  stream.emit(run.id, { type: 'run.completed', artifacts: refs });
}

// A PDF written at minute 23 of a 40-minute run sits on disk the
// whole time with no DB row, no event and no UI until finalizeRun()
// fires. If the run is stopped or crashes, it never fires.
```

AFTER server/artifact-registry.ts

```ts
// AFTER: every write registers itself the moment it lands.
export async function registerArtifact(
  run: Run, path: string, bytes: Buffer, meta: WriteMeta,
): Promise<Artifact> {
  const hash = sha256(bytes);
  const prev = await db.artifacts.findUnique({ where: { runId_path: { runId: run.id, path } } });
  if (prev?.contentHash === hash) return prev;             // same content -> no event, no version

  const artifact = await db.artifacts.upsert({
    where:  { runId_path: { runId: run.id, path } },
    create: {
      runId: run.id, path, name: basename(path), kind: kindOf(path),
      bytes: bytes.length, version: 1, status: 'ready',
      producedBy: meta.producer, contentHash: hash,
      msIntoRun: Date.now() - run.startedAt.getTime(),
    },
    update: { bytes: bytes.length, version: { increment: 1 }, status: 'ready', contentHash: hash },
  });

  emitDebounced(run.id, artifact.id, {                     // 500ms per artifact
    type: artifact.version === 1 ? 'artifact.created' : 'artifact.updated',
    artifact: publicView(artifact),
  });
  return artifact;
}

// The ONLY way a file gets written: main agent, sub-agents, background tasks, plugins.
export async function writeWorkspaceFile(run: Run, path: string, bytes: Buffer, meta: WriteMeta) {
  await markWriting(run.id, path);                         // row -> 'writing' (pill shows "Writing...")
  await workspace.writeAtomic(run.id, path, bytes);        // tmp + rename: never a half-written PDF
  return registerArtifact(run, path, bytes, meta);         // row -> 'ready' (openable mid-run)
}

// Run end: nothing is collected, only status flips.
                                                                                failed') {
```

Every writer goes through `writeWorkspaceFile()` — main agent, sub-agents, background tasks, plugin tools that produce attachments. If today there are several write paths, consolidating them is task 1; the panel can't be trusted until it is.

```ts
  ...
  const ids = (await db.artifacts.findMany({ where: { runId: run.id }, select: { id: true } })).map(r =>
r.id);
  stream.emit(run.id, { type: 'artifact.finalized', artifactIds: ids });
  stream.emit(run.id, outcome === 'completed'
    ? { type: 'run.completed' }
    : { type: 'run.failed', error: run.error ?? 'stopped by user' });
}
```

6.5 API

BEFORE — server/routes/runs.ts

```
// TODAY: artifacts ride on the run object, empty until completed.
router.get('/runs/:id', auth, async (req, res) => {
  const run = await db.runs.findUniqueOrThrow({ where: { id: req.params.id } });
  res.json({
    ...run,
    artifacts: run.status === 'completed' ? run.artifacts : [],
  });
});
```

AFTER — server/routes/artifacts.ts

```
// AFTER: run-scoped list that includes drafts; content via a short-lived signed URL.
router.get('/runs/:id/artifacts', auth, async (req, res) => {
  await assertCanReadRun(req.user, req.params.id);
  const rows = await db.artifacts.findMany({
    where: { runId: req.params.id },
    orderBy: { createdAt: 'asc' },
  });
  res.json({ artifacts: rows.map(publicView) });          // refresh mid-run -> panel repopulates
});

router.get('/artifacts/:id/url', auth, async (req, res) => {
  const a = await db.artifacts.findUniqueOrThrow({ where: { id: req.params.id } });
  await assertCanReadRun(req.user, a.runId);              // same auth as finals; drafts get no shortcut
  const url = await storage.signedUrl(objectKey(a), { ttlSeconds: 600, version: a.version });
  res.json({ url, version: a.version, status: a.status });
});
```

6.6 Frontend hook

BEFORE — web/hooks/useRunArtifacts.ts

```
// TODAY
export function useRunArtifacts(runId: string): Artifact[] {
  const run = useRun(runId);
  return run?.status === 'completed' ? run.artifacts : [];  // [] for the entire run
}
```

AFTER web/hooks/useRunArtifacts.ts

```ts
// AFTER: seed from REST (survives refresh), then live-upsert from the stream.
export function useRunArtifacts(runId: string): Artifact[] {
  const [byId, setById] = useState<Record<string, Artifact>>({});

  useEffect(() => {
    let alive = true;
    api.get<{ artifacts: Artifact[] }>(`/runs/${runId}/artifacts`)
       .then(r => { if (alive) setById(indexBy(r.artifacts, 'id')); });

    const off = runStream.subscribe(runId, (ev: RunEvent) => {
      switch (ev.type) {
        case 'artifact.created':
        case 'artifact.updated':
          setById(prev => ({ ...prev, [ev.artifact.id]: ev.artifact }));
          break;
        case 'artifact.finalized':
          setById(prev => mapValues(prev, a =>
            ev.artifactIds.includes(a.id) && a.status !== 'partial' ? { ...a, status: 'final' } : a));
          break;
      }
    });
    return () => { alive = false; off(); };
  }, [runId]);

  return useMemo(
    () => Object.values(byId).sort((a, b) => a.createdAt.localeCompare(b.createdAt)),
    [byId],
  );
}
```

6.7 Panel component

NEW — web/playground/OutputsPanel.tsx

```tsx
export function OutputsPanel({ runId }: { runId: string }) {
  const outputs = useRunArtifacts(runId);
  const [open, setOpen] = useRightRail('outputs');           // persisted per user
  const unseen = useUnseenCount(outputs, open);               // badge while collapsed
  const announce = useLiveRegion('polite');

  useEffect(() => {
    const last = outputs.at(-1);
    if (last?.status === 'ready') announce(`New output ready: ${last.name}`);
  }, [outputs, announce]);

  if (!open) {
    return <RailToggle icon={OutputsIcon} label="Outputs" badge={unseen} onClick={() => setOpen(true)} />;
  }

  return (
    <aside className="od-outputs" aria-label="Outputs">
      <header className="od-outputs__hd">
        <h2>Outputs {outputs.length > 0 && <span className="od-count">{outputs.length}</span>}</h2>
        <button className="od-iconbtn" aria-label="Create an output" title="Create an output"
                onClick={openOutputTemplates}>+</button>
      </header>

      {outputs.length === 0 ? (
        <p className="od-outputs__empty">Files your agent creates appear here while it works.</p>
      ) : (
        <ul className="od-outputs__list">
          {outputs.map(a => <OutputRow key={a.id} artifact={a} />)}
        </ul>
      )}

      {flags.outputsSources && (
        <section className="od-outputs__sources" aria-label="Sources">      {/* phase 2 */}
          <h3>Sources</h3>
          <button className="od-iconbtn" aria-label="Add a source">+</button>
        </section>
      )}
      <a className="od-outputs__all" href="/outputs">All outputs →</a>
    </aside>
  );
}

function OutputRow({ artifact: a }: { artifact: Artifact }) {
  const open = async () => {
    const { url } = await api.get<{ url: string }>(`/artifacts/${a.id}/url`); // current version
    previewPane.open({ url, title: a.name, kind: a.kind });                   // run keeps streaming
    track('output_opened', { runId: a.runId, artifactId: a.id, status: a.status, msIntoRun: a.msIntoRun });
  };
  return (
    <li className={`od-output od-output--${a.status}`}>
      <KindIcon kind={a.kind} />
      <button className="od-output__main" onClick={open}>
        <span className="od-output__name">{a.name}</span>
        <span className="od-output__meta">
          <StatusPill status={a.status} /> · {fmtMinutes(a.msIntoRun)} into run · {fmtBytes(a.bytes)}
          {a.status === 'writing' && ` · v${a.version}`}
        </span>
      </button>
      <RowActions artifact={a} />   {/* Open · Download · ⋯ (Copy link, Open in Outputs) */}
    </li>
```

```
  );
}
```

NEW web/playground/outputs-panel.css

```css
/* Uses existing Playground tokens; the hex fallbacks are the values sampled in §5.3. */
.od-outputs {
  width: 320px; padding: 16px; display: flex; flex-direction: column; gap: 12px;
  background: var(--od-surface-0, #ffffff); border-left: 1px solid var(--od-line-1, #e4e7ec);
  overflow-y: auto;
}
.od-outputs__hd { display: flex; align-items: center; justify-content: space-between; }
.od-outputs__hd h2 { margin: 0; font: 600 15px/1.2 var(--od-font); color: var(--od-ink-700, #5d5d5d); }
.od-count {
  margin-left: 6px; padding: 3px 7px; border-radius: 999px;
  font: 500 12px/1 var(--od-font); color: var(--od-brand-700, #326e5c); background: var(--od-brand-50,
#f1f9f6);
}
.od-iconbtn {
  width: 28px; height: 28px; border-radius: 8px; background: var(--od-surface-0, #fff);
  border: 1px solid var(--od-line-1, #e4e7ec); color: var(--od-ink-500, #8d8d8d); cursor: pointer;
}
.od-iconbtn:hover { border-color: var(--od-brand-500, #52aa8b); color: var(--od-brand-700, #326e5c); }
.od-iconbtn:focus-visible { outline: 2px solid var(--od-brand-500, #52aa8b); outline-offset: 2px; }

.od-outputs__empty { margin: 0; font: 400 13px/1.5 var(--od-font); color: var(--od-ink-500, #8d8d8d); }
.od-outputs__list { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; gap:
4px; }

.od-output {
  display: grid; grid-template-columns: 20px 1fr auto; gap: 10px; align-items: center;
  min-height: 44px; padding: 8px; border-radius: 10px;
  animation: od-enter 160ms ease-out;
}
.od-output:hover, .od-output:focus-within { background: var(--od-surface-1, #f6f7f8); }
.od-output__main { all: unset; cursor: pointer; display: grid; gap: 2px; min-width: 0; }
.od-output__name {
  font: 500 13px/1.3 var(--od-font); color: var(--od-ink-900, #2e2e2e);
  overflow: hidden; text-overflow: ellipsis; white-space: nowrap;
}
.od-output__meta { font: 400 12px/1.4 var(--od-font); color: var(--od-ink-700, #5d5d5d); } /* not #8d8d8d:
3.3:1 fails AA */

.od-pill { display: inline-flex; align-items: center; gap: 5px; padding: 1px 7px; border-radius: 999px;
font-weight: 500; }
.od-pill--writing { background: var(--od-brand-50, #f1f9f6); color: var(--od-brand-700, #326e5c); }
.od-pill--writing::before {
  content: ''; width: 6px; height: 6px; border-radius: 50%;
  background: var(--od-brand-500, #52aa8b); animation: od-pulse 1.2s ease-in-out infinite;
}
.od-pill--ready   { background: var(--od-brand-100, #e0edea); color: var(--od-brand-700, #326e5c); }
.od-pill--final   { background: var(--od-chip-bg, #e7e7e7);   color: var(--od-ink-700, #5d5d5d); }
.od-pill--partial { background: #fbf5e6; color: #7a5f1c; }

.od-rail-badge {
  position: absolute; top: -6px; right: -6px; min-width: 15px; height: 15px; padding: 0 4px;
  border-radius: 999px; background: var(--od-brand-400, #61b597); color: #fff;
  font: 800 10px/15px var(--od-font); text-align: center; box-shadow: 0 0 0 2px var(--od-surface-0, #fff);
}

@keyframes od-enter { from { opacity: 0; transform: translateY(4px); } to { opacity: 1; transform: none; } }
@keyframes od-pulse { 0%, 100% { opacity: 1; } 50% { opacity: .35; } }
@media (prefers-reduced-motion: reduce) {
  .od-output { animation: none; }
  .od-pill--writing::before { animation: none; }
}
```

6.8 "+" → templates tray and pre-filled composer

NEW web/playground/output-templates.ts

```ts
export interface OutputTemplate {
  id: string;
  title: string;
  skill: 'documents' | 'spreadsheets' | 'slides' | 'web';
  kind: Artifact['kind'];
  thumbnail: string;         // rendered preview of a real example output, 296x372 @2x
  prompt: string;            // lands in the composer; the user just presses send
}

export const OUTPUT_TEMPLATES: OutputTemplate[] = [
  { id: 'design-report', title: 'Design report', skill: 'documents', kind: 'docx',
    thumbnail: '/tpl/design-report.png',
    prompt: 'Create a design report with the Documents skill. Start by asking me what it should be about.'
},
  { id: 'investigation-report', title: 'Investigation report', skill: 'documents', kind: 'pdf',
    thumbnail: '/tpl/investigation-report.png',
    prompt: 'Create an investigation report with the Documents skill. Start by asking me what it should
cover.' },
  { id: 'ic-memo', title: 'Investment committee memo', skill: 'documents', kind: 'docx',
    thumbnail: '/tpl/ic-memo.png',
    prompt: 'Create an investment committee memo with the Documents skill. Start by asking me about the
deal.' },
  { id: 'evidence-workbook', title: 'Evidence workbook', skill: 'spreadsheets', kind: 'xlsx',
    thumbnail: '/tpl/evidence-workbook.png',
    prompt: 'Create an evidence workbook with the Spreadsheets skill. Start by asking me what to track.' },
  { id: 'letterhead', title: 'Minimal letterhead', skill: 'documents', kind: 'docx',
    thumbnail: '/tpl/letterhead.png',
    prompt: 'Create a letter on a minimal letterhead with the Documents skill. Start by asking me who it is
for.' },
  { id: 'deck', title: 'Slide deck', skill: 'slides', kind: 'pptx',
    thumbnail: '/tpl/deck.png',
    prompt: 'Create a slide deck with the Slides skill. Start by asking me who the audience is.' },
];

// "+" behaviour, mirroring ChatGPT: tray opens, composer is pre-filled, nothing is sent.
export function openOutputTemplates() {
  composer.setDraft({
    text:   'Create a new document with ',
    chips:  [{ skill: 'documents', label: 'Documents' }],   // brand-50 bg, brand-700 text
    suffix: '. Start by asking me what it should be about.',
    focus:  'end',
  });
  composer.showTray({
    kind: 'templates',
    items: OUTPUT_TEMPLATES,
    onPick: t => {
      composer.setDraft({ text: t.prompt, chips: [{ skill: t.skill, label: labelFor(t.skill) }], focus:
'end' });
      track('template_picked', { templateId: t.id });
    },
    onClose: () => composer.hideTray(),
  });
  track('outputs_plus_clicked');
}
```

6.9 Lifecycle and edge cases

CASE	BEHAVIOUR
Run completes	All rows → `final`; one `artifact.finalized` event; `run.completed` no longer carries a file list
User presses ■ / run fails	Rows in `writing` or `ready` → `partial`. Files are kept. Panel keeps showing them with the Partial pill
Agent rewrites the same path	Same row, `version++`, `artifact.updated`. Identical content hash → no event, no version bump
Chatty writer (e.g. 200 saves in 10s)	`emitDebounced` coalesces per artifact at 500ms; the DB still records every version
Half-written file	Never visible: writes are tmp + atomic rename, and the row flips to `ready` only after the tool returns. Opening a `writing` row serves the last complete version
Refresh mid-run	`GET /runs/:id/artifacts` seeds the panel including drafts; the stream resumes from the last event id
Multiple runs in a session	Panel lists the active run first, earlier runs collapsed under their timestamp
Auth	Signed URLs for drafts use exactly the run's read permission; TTL 10 min; no shortcut for "temporary" files
Large files	Preview for pdf/html/md/image/csv; download-only above 25 MB or for pptx/xlsx/docx if no preview renderer exists
Sub-agents / background tasks	Same `registerArtifact()`; `producedBy` carries the task id so the row can say "from background task"

6.10 Acceptance criteria — definition of done

1. A file written at T+23 min appears in the panel within 2 s of the write, while the run continues
2. Clicking it opens the current version; the run stream is not interrupted, the to-do list keeps updating
3. Refresh at T+24 min: panel repopulates with all rows including `writing` ones
4. Rewriting the same path 10× produces one row at v10, not ten rows
5. Pressing ■ at T+25 min keeps every file; rows show Partial; files open
6. Run completion flips all rows to Final with one event
7. Collapsed rail badge increments on each new `ready` row and clears on open
8. "+" opens the tray and pre-fills the composer; nothing is sent until send; picking a card swaps the prompt
9. Dark mode, keyboard-only navigation, and screen-reader announcement ("New output ready: …") all pass

0. 200 writes in 10 s → ≤ 20 stream events for that artifact, UI stays at 60 fps

6.11 Telemetry

- `outputs_panel_opened` {runId, trigger: toggle | badge | auto}
- `output_opened` {runId, artifactId, status, msIntoRun, kind}
- `outputs_plus_clicked`, `template_picked` {templateId}, `template_prompt_sent`
- `time_to_first_output_ms` per run — the number that shows whether the panel changes behaviour

6.12 Engineer to-dos

Backend

- [ ] Migration in §6.2; backfill existing rows as `final`, v1
- [ ] Consolidate all file-write paths into `writeWorkspaceFile()` (main agent, sub-agents, background tasks, plugins)
- [ ] `registerArtifact()` with content hash, version bump, `msIntoRun`
- [ ] Emit `artifact.created / updated / finalized` on the run stream; 500ms per-artifact debounce
- [ ] `GET /runs/:id/artifacts` and `GET /artifacts/:id/url` (signed, drafts included, run-scoped auth)
- [ ] Finalize on complete; partial on stop/fail; nothing deleted
- [ ] Feature flag `outputs_panel`; load test AC 10

Frontend

- [ ] `useRunArtifacts`: REST seed + stream upsert (§6.6)
- [ ] `OutputsPanel` + `OutputRow` + `StatusPill` + `KindIcon` (§6.7), tokens from §5.3
- [ ] Rail toggle with unseen badge; persist open/closed per user; default rule from §5.1
- [ ] Preview pane opens without tearing down the stream subscription
- [ ] "+" → templates tray + composer pre-fill with skill chip (§6.8); `OUTPUT_TEMPLATES` config + thumbnails from design
- [ ] Live region + toast when collapsed; reduced-motion paths
- [ ] Telemetry events (§6.11); Playwright tests for AC 1–9

§7 · ROLLOUT & ESTIMATES

Two weeks calendar with three people in parallel; the flag lets phase 1 ship before templates are pretty

PHASE	SHIPS	EFFORT (WORKING DAYS)	ASSUMES
1 · This spec	Live registration, panel, toggle + badge, preview mid-run, "+" with tray and pre-fill	BE 3–4 · FE 4–5 · Design 2 · QA 1–2	One write path (or one day to consolidate); SSE/WebSocket stream exists (the to-do list already updates live); storage already mints signed URLs (your run's to-do says so)
2	Sources section; version history per output; share link; "from background task" attribution	FE 3 · BE 2	Phase 1 telemetry shows outputs get opened mid-run
3	Outputs library (left nav) merges "Artifacts": search, filter by kind/run, pin	FE 4 · BE 2	Naming decision taken (§8)

- **Quick math:** phase 1 = 10–14 person-days; with BE, FE and design in parallel that is ~8–10 calendar days plus a few days of flag rollout → ~2 weeks
- **Sensitivity:** the two drivers are (a) number of existing write paths to consolidate (each extra path ≈ +0.5–1 day) and (b) whether a preview renderer exists for pdf/html (none → +2 days or download-only in phase 1)
- **Order:** backend first, behind the flag, with the panel reading from REST only — then wire the stream. Templates last; they're independent

§8 · ASSUMPTIONS I MADE, AND THREE DECISIONS FOR YOU

Everything below is a guess from two screenshots; correct any line and the rest still holds

Assumptions

- You wrote "left hand" — I read it as "side panel" and mirrored ChatGPT's right rail because the collapsed-rail toggle already sits top-right in the Playground. If you meant the left nav, the same panel mounts there as a drawer and nothing else changes
- Stack: React + TypeScript, a live run stream (the to-do card updates in place, so one exists), a workspace/file store that can sign URLs (your run's to-do: "verify/mint signed URLs… HTTP 206")
- All agent file writes can be routed through one function. If not, that consolidation is the first engineering task and the biggest estimate risk
- Colours were sampled from a Retina screenshot; treat them as ±1–2 steps from the real tokens
- Template set is a first cut. The six titles mirror ChatGPT's mix plus OnDemand's investigation and workbook outputs; the final set is a design/product call
- No versioning UI, sources, or cross-run library in phase 1

Decisions for Mk

DECISION	RECOMMENDATION	ALTERNATIVE
1 · Placement	**Right rail**, toggle in the existing slot — mirrors ChatGPT, transcript stays central	Left-nav drawer: keeps the transcript full width but hides the panel behind a click
2 · Naming	**"Outputs"** everywhere: panel, left nav, library page. Rename "Artifacts"	Keep "Artifacts" for the library, "Outputs" for the panel — two words for one thing
3 · Default open	**Auto-open at ≥1280px** when the first output lands; collapsed with badge below	Always collapsed until clicked — safer for the transcript, weaker discovery

§9 · RISKS, TRADE-OFFS, COUNTER-ARGUMENTS

The panel costs transcript width and invites event storms; both are cheap to control

Risks and trade-offs

- **Width:** 320px off the transcript on a 13" laptop. Mitigation: collapsed by default below 1280px, badge keeps discovery
- **Event storms:** agents that save every few seconds. Mitigation: content hash + 500ms debounce; DB still has every version
- **Corrupt mid-write opens:** a PDF opened while being written. Mitigation: atomic rename + `ready` only after the tool returns
- **Security:** signed URLs for drafts widen the window in which files are reachable. Mitigation: same run-read auth, 10-min TTL, no public links in phase 1
- **Naming drift:** "Artifacts" and "Outputs" coexisting confuses users. Mitigation: decision 2 in §8, taken before design handoff
- **Estimate risk:** hidden write paths. Mitigation: BE day 1 is an inventory of every place a file gets written

Counter-arguments

- **"Just have the agent post a link in the transcript when it writes a file."** Cheaper, and worth doing too — but it scrolls away, and it doesn't survive a stopped run. The panel is a stable index; the link is a notification
- **"Mid-run files are unstable; wait for the end."** The 23-minute case is the cost of that stance. Status pills and atomic writes make instability visible instead of hidden
- **"Copy ChatGPT exactly, users already know it."** Copy the pattern (§4), not the palette or the wording — OnDemand's green, one word "Outputs", and status pills ChatGPT doesn't have
- **"Templates are scope creep."** They're the smallest part (1–2 days) and they answer a different problem — the blank Playground — which is why ChatGPT put them behind the same "+"